

**Гимназија „Светозар Марковић”, Ниш**

# **МАТУРСКИ РАД**

**Предмет:**

**Рачунарство и информатика**

**Тема:**

**Електронски дневник за праћење школских обавеза**

**Ментор:**

**Зорана Јанчић**

**Ученик:**

**Вукашин Влајић**

Ниш, мај 2025.

## Садржај

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| 1. Увод.....                                                          | 3  |
| 2. Фронтенд.....                                                      | 4  |
| 2.1. Почетна сцена - „MainMenu”.....                                  | 5  |
| 2.2. Сцена - „MainMenuProfesor“.....                                  | 7  |
| 2.3. Сцене за пријављивање - „LoginMenu“ и „LoginProfesor“.....       | 8  |
| 2.4. Сцене за регистрацију - „RegisterMenu“ и „RegisterProfesor“..... | 9  |
| 3. Бекенд.....                                                        | 10 |
| 3.1. Регистрација и пријава корисника.....                            | 11 |
| 3.2. Додавање и преглед оцена.....                                    | 14 |
| 3.3. Додавање и преглед обавеза.....                                  | 17 |
| 3.4. Преглед месечног извештаја.....                                  | 18 |
| 4. Закључак.....                                                      | 19 |
| Литература.....                                                       | 20 |

# 1. Увод

У савременом добу, информације представљају један од највреднијих ресурса. Са порастом количине података које стварамо и користимо сваког дана, јавља се потреба за ефикасним чувањем, организовањем и приступањем тим подацима. Управо ту на сцену ступају базе података – темељ сваког информационог система.

Базе података су организовани скупови података који омогућавају њихово систематско чување и лако управљање. Захваљујући базама података, могуће је да велики број корисника истовремено приступи информацијама, да се подаци ажурирају у реалном времену и да се обезбеди њихов интегритет и безбедност. Ове особине чине базе података кључним елементом у свим областима које захтевају поуздано чување и обраду података.

Постоје различите врсте база података, као што су релационе базе података које користе табеле за организовање података, као и не-релационе базе (NoSQL) које су погодне за рад са великим обимом различитих и често неструктурираних података. Савремени системи често комбинују више врста база података како би постигли најбоље перформансе и флексибилност.

Примена база података је изузетно широка – од банкарских система који управљају финансијским трансакцијама, преко здравствених информационих система који чувају медицинске евиденције, образовних платформи које прате напредак ученика, до друштвених мрежа и електронске трговине где се обрађују огромне количине података о корисницима и производима.

У овом матурском раду биће представљена апликација која је израђена на темељу базе података. Кроз рад ће бити објашњен процес креирања и дизајнирања базе података, начин повезивања базе са корисничким интерфејсом, као и имплементација функција које омогућавају унос, измену, брисање и приказ података.

Циљ рада је да се прикаже колико су базе података важне за ефикасан рад савремених апликација, као и да се теоријска знања примене у практичном пројекту који илуструје њихову употребу у реалним условима.

## 2. Фронтенд

Фронтенд апликације реализован је у оквиру Unity окружења, користећи програмски језик C#. Кориснички интерфејс (UI) обухвата све елементе са којима корисник директно комуницира – као што су дугмад, поља за унос података, менији и обавештења. У Unity-ју су ови елементи дизајнирани коришћењем Canvas система и UI компоненти (нпр. Button, InputField, Text).

Логика понашања интерфејса, као што је реаговање на клик корисника, валидација унетих података и слање података серверу, реализована је у C# скриптама. На тај начин обезбеђено је да је корисничко искуство интуитивно и функционално, а све акције корисника се прослеђују серверу или обрађују унутар апликације.

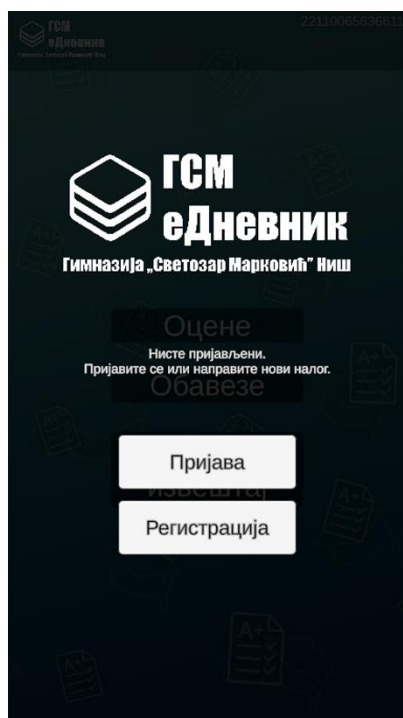
Апликација је организована кроз шест различитих сцена у Unity-ју, од којих је свака задужена за посебан део функционалности и корисничког искуства. Сцене су:

- „LoginMenu“ – сцена за пријаву ученика,
- „LoginProfesor“ – сцена за пријаву професора,
- „RegisterMenu“ – сцена за регистрацију ученика,
- „RegisterProfesor“ – сцена за регистрацију професора,
- „MainMenu“ – главни мени након пријаве ученика,
- „MainMenuProfesor“ – главни мени након пријаве професора.

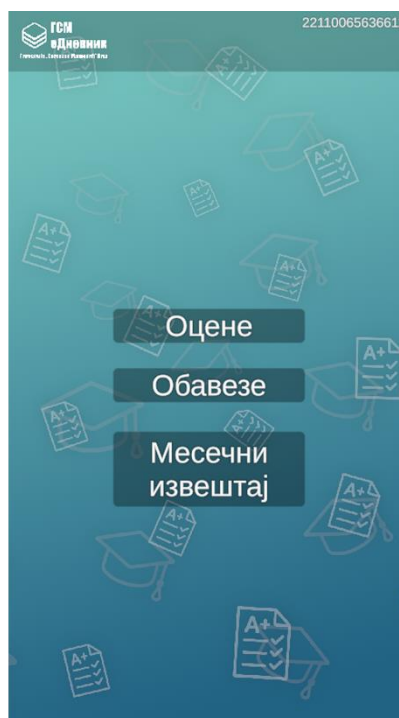
Свака сцена има прилагођен кориснички интерфејс у зависности од типа корисника (ученик или професор) и садржи елементе попут поља за унос (нпр. корисничко име и лозинка), дугмади за потврду, као и визуелна обавештења о статусу (успешна пријава, грешка и сл.).

Оваква организација сцена омогућава јасну поделу улога и функција унутар апликације, чиме се побољшава прегледност и одрживост кода, као и корисничко искуство.

## 2.1. Почетна сцена - „MainMenu”



слика 2.1



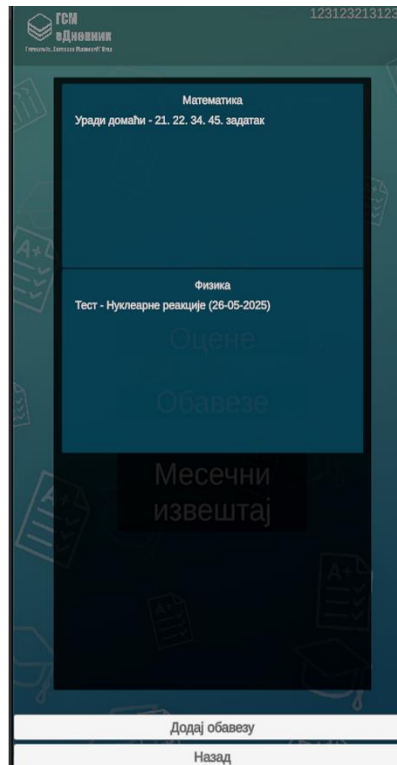
слика 2.2

При покретању апликације, кориснику се приказује почетни панел са могућностима пријаве или регистрације (слика 2.1). Уколико корисник већ поседује налог, може се пријавити и тиме добити приступ функционалностима апликације. У супротном, може изабрати опцију за креирање новог налога.

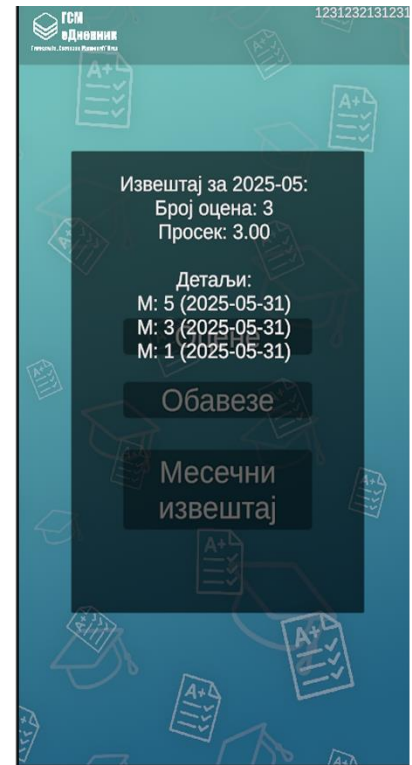
Уколико се корисник пријави као ученик, апликација га враћа на сцену „MainMenu“, али овог пута са деактивираним почетним панелом (слика 2.2). На тај начин кориснику се омогућава директан приступ функционалностима намењеним ученицима, као што су преглед оцена (слика 2.3), школских обавеза (слика 2.4) и месечних извештаја (слика 2.5)



слика2.3



слика 2.4



слика 2.5

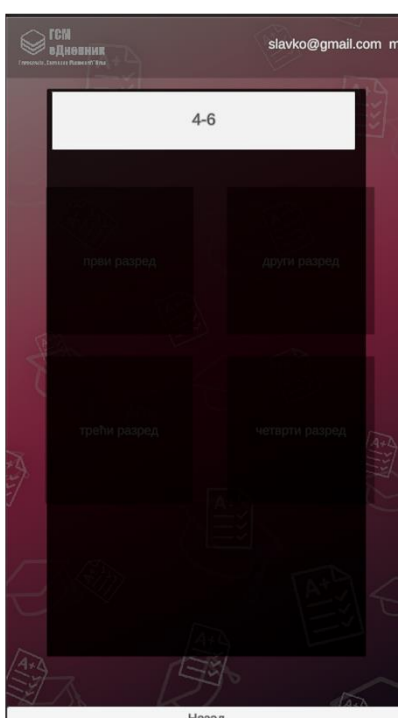
## 2.2. Сцена - „MainMenuProfesor“

Након пријаве као професор, кориснику се учита сцена „MainMenuProfesor“, прилагођена искључиво наставном особљу. У овој сцени професору је омогућен преглед свих разреда и одељења којима предаје, као и увид у ученике у тим одељењима. Систем ограничава приступ – професор не може видети нити оцењивати ученике у одељењима у којима не предаје.

На почетном екрану налазе се четири дугмета (слика 2.6), свако посвећено једном разреду (први до четврти). Кликом на одговарајући разред, отвара се избор одељења (слика 2.7), након чега професор може одабрати ученика ком жели да додели оцену. Након избора ученика, приказује се интерфејс за унос оцене, која се затим бележи у базу података (слика 2.8).



слика 2.6



слика 2.7

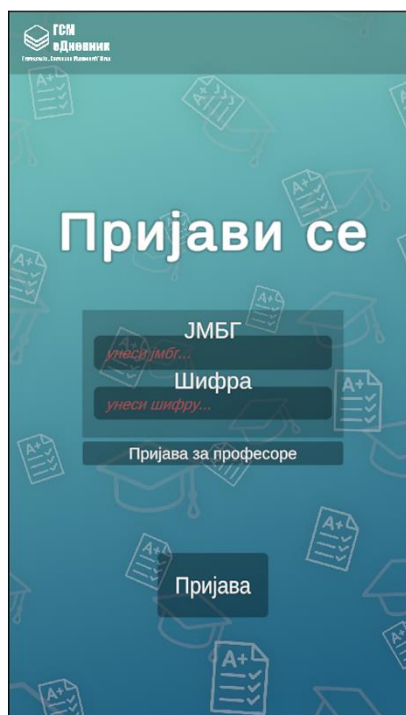


слика 2.8

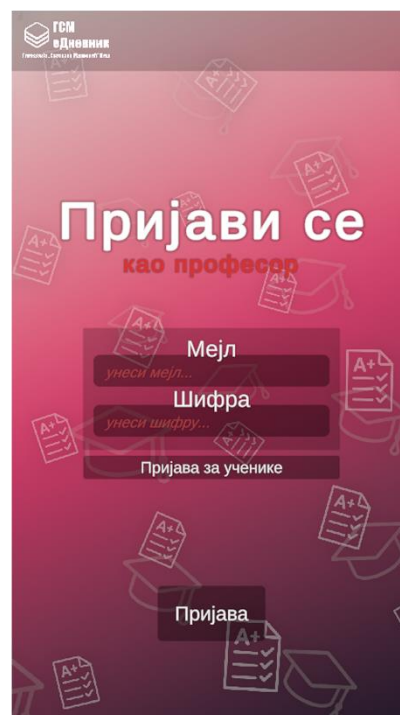
## 2.3. Сцене за пријављивање - „LoginMenu“ и „LoginProfesor“

Уколико корисник изабере опцију „Пријава“ са почетног екрана, апликација га преусмерава на сцену „LoginMenu“, намењену пријави ученика (слика 2.9). У овој сцени корисник уноси свој ЈМБГ и своју лозинку, након чега, кликом на дугме за пријаву, апликација проверава унете податке и, уколико су исправни, преусмерава ученика на сцену „MainMenu“.

Уколико корисник жели да се пријави као професор, може то учинити кликом на посебно дугме које га води у сцену „LoginProfesor“ (слика 2.10). Тамо на сличан начин уноси своје акредитиве, а након успешне пријаве добија приступ сцени „MainMenuProfesor“, где му је омогућен рад са одељењима и ученицима којима предаје.



слика 2.9



слика 2.10

## 2.4. Сцене за регистрацију - „RegisterMenu“ и „RegisterProfesor“

Слично сценама за пријављивање, сцена „RegisterMenu“ (слика 2.11) се учитава када корисник са почетног екрана изабере опцију „Регистрација“. Ова сцена је намењена регистрацији ученика и садржи поља за унос свих неопходних података.

Уколико корисник жели да се региструје као професор, може то учинити кликом на посебно дугме које га из сцене „RegisterMenu“ води у сцену „RegisterProfesor“ (слика 2.12). Тамо попуњава одговарајуће податке предвиђене за наставно особље.

Након попуњавања свих поља, регистрација се врши кликом на дугме. Пре уписа новог корисника у базу, систем проверава да ли већ постоји корисник са истим ЈМБГ-ом, односно мејл адресом, за професоре, чиме се спречава дупла регистрација.

слика 2.11

слика 2.12

### 3. Бекенд

Бекенд апликације реализован је коришћењем више технологија, прилагођених различитим аспектима комуникације и управљања подацима.

За комуникацију између едитора и позадинског система коришћен је C#, који обезбеђује брз и поуздан пренос података у реалном времену. За интеракцију са базом података на серверу примењен је PHP, захваљујући својој флексибилности и широкој подршци за веб развој. Структура и креирање табела у бази реализовани су помоћу MySQL система за управљање релационим базама података, који обезбеђује ефикасно складиштење и приступ информацијама.

У фази развоја, сервер је локално смештен помоћу МАРМ алата, што омогућава висок ниво безбедности и потпуну контролу над радом система током тестирања. Такав приступ обезбеђује да се подаци чувају искључиво локално, чиме се минимизује ризик од неовлашћеног приступа.

По завршетку развоја, прелазак на продукционо окружење је једноставан, јер је структура система осмишљена тако да омогућава лаку миграцију сервера на cloud платформу, чиме се омогућава шира доступност и скалабилност апликације.

## 3.1. Регистрација и пријава корисника

Захтев за регистрацију корисника иницира се из C# кода, који шаље све неопходне податке серверу ради обраде. Ови подаци се прослеђују PHP скрипти која их валидацијом и обрадом уписује у MySQL базу.

```
private IEnumerator Register()
{
    // Креирање WWW форме која ће садржати податке за регистрацију
    WWWForm form = new WWWForm();

    // Додавање поља у форму (узимају се подаци из UI елемената)
    form.AddField("jmbg", jmbgField.text);
    form.AddField("ime", imeField.text);
    form.AddField("prezime", prezimeField.text);
    form.AddField("password", passwordField.text);
    form.AddField("razred", razredDropdown.value + 1); // +1 јер индекси крећу
од 0
    form.AddField("odeljenje", odeljenjeDropdown.value + 1); // +1 из истог
разлога

    // Слање POST захтева ка PHP скрипти на серверу
    WWW www = new WWW("http://localhost/sqlconnect/register.php", form);

    // Чека се да се одговор врати са сервера
    yield return www;

    // Обрада одговора са сервера
    if(www.text == "0")
    {
        Debug.Log("Uspešno poslat zahtev za registraciju");

        // Успешна регистрација -> учитај сцену
        UnityEngine.SceneManagement.SceneManager.LoadScene(0);
    }
    else
    {
        // Регистрација није успела -> испиши број грешке који долази из PHP
скрипте
        Debug.Log("User Creation Failed. Error #" + www.text);
    }
}
```

```
<?php
// Повезивање са MySQL базом података
$con = mysqli_connect("localhost","root","root","unityaccess");
if (mysqli_connect_errno())
{
    echo "1: connection failed"; // грешка #1: повезивање није успело
    exit();
}

// Преузимање података послатих преко POST методе
$jmbg = $_POST["jmbg"];
$ime = $_POST["ime"];
$prezime = $_POST["prezime"];
$password = $_POST["password"];
$razred = $_POST["razred"];
$odeljenje = $_POST["odeljenje"];

// Провера да ли већ постоји ученик са истим ЈМБГ-ом
$jmbgcheckquery = "SELECT jmbg FROM ucenici WHERE jmbg = '" . $jmbg .
"";";
$jmbgcheck = mysqli_query($con, $jmbgcheckquery) or die("2: jmbg check
query failed"); // грешка #2: упит за проверу ЈМБГ-а није успео

if(mysqli_num_rows($jmbgcheck) > 0)
{
    echo "3: jmbg already exists"; // грешка #3: ЈМБГ већ постоји у бази
    exit();
}

// Хеширање лозинке помоћу SHA-256 (crypt функција са salt-ом)
// Salt садржи 'idegas' и име ученика како би био јединствен
$salt = "\$5$rounds=5000\$" . "idegas" . $ime . "\$";
$hash = crypt($password, $salt);

// Унос новог ученика у базу
$insertuserquery = "INSERT INTO ucenici (jmbg, ime, prezime, hash, salt,
razred, odeljenje, verifikacija)
VALUES ( '" . $jmbg . "', '" . $ime . "', '" . $prezime . "', '" .
$hash . "', '" . $salt . "', '" . $razred . "', '" . $odeljenje . "', '" . 0
. "')";
mysqli_query($con, $insertuserquery) or die("4: insert ucenici query
failed"); // грешка #4: упит за унос није успео

// Повратна вредност 0 означава успешну регистрацију
echo("0");
?>
```

Након што је захтев за регистрацију успешно послат и обрађен, нови налог се евидентира у бази података, али са статусом неверификованог корисника. Да би се налог активирао, потребно је да администратор приступи бази података преко phpMyAdmin интерфејса и у колону верификација упише вредност 1 за дати налог.

Овај ручни корак у процесу верификације омогућава већу контролу над регистрованим корисницима и спречава неауторизовано пријављивање. Све док налог није верификован, систем не дозвољава приступ апликацији, чиме се значајно повећава ниво безбедности и интегритета платформе.

Процес пријављивања на налог реализован је на сличан начин као и регистрација. Корисник уноси своје податке у апликацију, након чега се они шаљу серверу путем С# и РНР комуникације. РНР скрипта затим врши проверу достављених података упоређујући их са постојећим записима у MySQL бази.

Уколико се унети подаци поклапају са подацима у бази и ако је налог претходно верификован (колона верификација има вредност 1), апликација одобрава приступ корисничком налогу. У супротном, пријава се одбија, чиме се обезбеђује сигурност и спречавају неауторизовани приступи.

## 3.2. Додавање и преглед оцена

Систем за евиденцију оцена имплементиран је у оквиру табеле ученик, где се све оцене чувају у једној колони у јединственом формату. Свака оцена представљена је као комбинација кода предмета, оцене и датирања, у следећем формату:

[код предмета][оцена]([датум]),

на пример: m4(2025-05-31)m3(2025-05-31).

У овом примеру, ученик је добио оцену 4 из предмета чији је код „m”, и то дана 2025-05-31, као и оцену 3 из истог предмета истог дана.

Кодови предмета дефинисани су у C# скрипти, где сваки предмет има јединствени идентификатор, који служи за препознавање предмета приликом уноса или приказа оцена. При читању или упису оцена, скрипта врши парсирање овог низа и по потреби га конвертује у читљив приказ (нпр. "Математика", "Физика") на основу унапред дефинисане мапе кодова.

```
// Речник који повезује кратке ознаке предмета са њиховим пуним називима
var subjectMap = new Dictionary<string, string>()
{
    {"s", "Српски језик и књижевност"},
    {"m", "Математика"},
    {"e", "Енглески језик"},
    {"fzc", "Физичко васпитање"},
    {"fiz", "Физика"},
    {"nem", "Немачки језик"},
    {"asa", "Анализа са алгебром"},
    //...
}
```

Додавање оцена врши се у PHP коду тако што се на већ постојеће оцене само дода нова.

```
...
$query = "SELECT ocene FROM ucenici WHERE jmbg = '$jmbg'";
$result = mysqli_query($con, $query) or die("2: query failed");
...
$row = mysqli_fetch_assoc($result);
$ocene = $row["ocene"];
...
$ocene .= $predmet . $ocena . "(" . $datum . ")";
$updateQuery = "UPDATE ucenici SET ocene = '$ocene' WHERE jmbg = '$jmbg'";
```

За преглед оцена у апликацији потребно је да се подаци из једне колоне „дешифрују“ и прикажу на јасан и прегледан начин.

Овај процес обавља се у C# коду, где се стринг са оценама раздваја на појединачне елементе. Након парсирања, подаци се форматирају у читљив облик и динамички приказују кориснику.

У Unity окружењу, за приказ се користи родитељски префаб<sup>1</sup> објекат који садржи три текстуална UI елемента (TextMeshPro). Свакој компоненти се додељује одговарајућа вредност.

```
. . .
oceneTemp = DBManager.ocene;
. . .
// Групе: 1=предмет, 2=оцена, 3=датум
var matches = Regex.Matches(oceneTemp, @"([a-zA-Z]+)(\d)\(([d\w-]+\))");

foreach (Match match in matches)
{
    // Издавање података из група регуларног израза
    string subjectCode = match.Groups[1].Value; // Код предмета, нпр. "mat"
    string grade = match.Groups[2].Value;       // Оцена, нпр. "5"
    string date = match.Groups[3].Value;        // Датум, нпр. "2025-05-31"

    // Прављење новог UI елемента из prefab-а за приказ једне оцене
    var instance = Instantiate(OcenaPref, ScrollViewContent);
    // Пронађи UI компоненте унутар prefab-а које ће приказати податке
    var subjectText =
instance.transform.Find("PredmetTXT").GetComponent<TextMeshProUGUI>();
    var gradeText =
instance.transform.Find("OcenaTXT").GetComponent<TextMeshProUGUI>();
    var dateText =
instance.transform.Find("DatumTXT").GetComponent<TextMeshProUGUI>();
    // Постави текст у UI компоненте
    gradeText.text = grade;
    dateText.text = date;
    // Ако речник предмета садржи код предмета, прикажи пуно име
    if (subjectMap.ContainsKey(subjectCode))
        subjectText.text = subjectMap[subjectCode];
    else
        // Ако није познат код, прикажи га као непознато са самим кодом
        subjectText.text = $"Unknown ({subjectCode})";
}
```

---

<sup>1</sup> Префаб (Prefab) је у Unity-ју унапред припремљени и сачувани објекат или група објеката са свим својствима и компонентама, који се може више пута користити и динамички инстанцирати у сцени.

Ради лакшег приступа оценама, оне се памте у C# класи „DBManager”.

```
public static class DBManager
{
    public static string ocene;
    . . .
    public static WWW GetOceneFromDB()
    {
        WWWForm form = new WWWForm();
        form.AddField("jmbg", jmbg);
        WWW www = new WWW("http://localhost/sqlconnect/extract_info.php",
form);
        return www;
    }
    public static IEnumerator UpdateOcene()
    {
        WWW www = GetOceneFromDB();
        yield return www;
        ocene = www.text;
    }
}
```

### 3.3. Додавање и преглед обавеза

Додавање обавеза у систему функционише на сличан начин као и додавање оцена. У овом случају, корисник – односно ученик – додаје нову обавезу тако што креира нови ред у табели „ucenik\_obaveza”. Та табела повезује конкретну обавезу са ЈМБГ-ом ученика, чиме се јасно дефинише којем ученику припада која обавеза.

Један ученик може имати више обавеза, тако да табела омогућава вишеструке уносе за истог ученика, сваки са различитим текстом обавезе и предметом на који се односи. Овим приступом се постиже флексибилност и прегледност у праћењу свих школских обавеза у оквиру апликације.

Када је потребно приказати обавезе ученика, подаци се читају из базе и шаљу клијентској апликацији у JSON<sup>2</sup> формату. За разлику од дела система који приказује оцене (где се користи једноставан текстуални одговор), овде се користи JSON.

Важно је нагласити да крајњи корисник не види сам JSON, већ се ти подаци аутоматски дешифрују и прикажу у пријатном графичком облику унутар апликације. На пример, у Unity-ју, свака обавеза је представљена као визуелни елемент (префаб), који садржи текст задатка и назив предмета. На тај начин, подаци се обрађују у позадини, а корисник види уредно приказане обавезе.

```
<?php
. . .
$selectquery = "SELECT tekst, predmet FROM ucenik_obaveza WHERE id_ucenik =
'$jmbg'";
$result = mysqli_query($con, $selectquery);
. . .
// Креирање празног низа који ће чувати све обавезе ученика
$obaveze = array();
// Пролажење кроз резултате и додавање сваке обавезе у низ
while($row = mysqli_fetch_assoc($result))
{
    $obaveze[] = array(
        "tekst" => $row['tekst'],      // Текст обавезе
        "predmet" => $row['predmet']  // Предмет на који се обавеза односи
    );
}

// Слање података назад клијенту у JSON формату
echo json_encode(["items" => $obaveze]);
. . .
?>
```

---

<sup>2</sup> JSON (JavaScript Object Notation) је лалан формат за размену података између сервера и клијента. Омогућава слање сложених структура (попут листа и објеката) у читљивом облику. У овом случају, обавезе се враћају као листа објеката са пољима „tekst“ и „predmet“.

## 3.4. Преглед месечног извештаја

Извештај се формира на основу података који се налазе у класи „DBManager”, тачније у стрингу „DBManager.ocene”, који садржи све оцене пријављеног ученика у унапред дефинисаном формату.

Програм користи регуларни израз (Regex)<sup>3</sup> да би издвојио појединачне оцене, предмет и датум из стринга. Затим се проверава да ли је оцена уписана у току текућег месеца — та провера се врши тако што се датум упоређује са тренутним месецом у формату уууу-ММ.

Уколико су пронађене оцене за текући месец, скрипта израчунава просек, број оцена, и формира детаљан преглед сваке оцене по предмету. На крају, све ове информације се уписују у TextMeshProUGUI елемент на екрану, тако да ученик добије прегледан извештај у оквиру апликације.

Уколико у датом месецу нема оцена, кориснику се приказује обавештење да не постоје унети подаци за тај период.

```
private IEnumerator GenerateIzvestaj()
{
    // Позива функцију која асинхроно ажурира податке о оценама из базе
    yield return StartCoroutine(DBManager.UpdateOcene());
    // Преузима све оцене као један стринг
    string oceneTemp = DBManager.ocene;
    // Узима тренутни месец у формату "уууу-ММ" ради филтрирања
    string targetMonth = DateTime.Now.ToString("уууу-ММ");

    var matches = Regex.Matches(oceneTemp, @"([a-zA-Z]+)(\d)\(((\d\[-]+\d)\)");
    . . .
    string datum = match.Groups[3].Value;
    // Проверава да ли датум почиње тренутним месецом
    if (datum.StartsWith(targetMonth))
    {
        // Ако је из тренутног месеца, додаје оцену у листу
        oceneUMesecu.Add(ocena);
        // Формира детаљан опис и додаје га у листу детаља
        detalji.Add($"{predmet.ToUpper()}: {ocena} ({datum})");
    }
    . . .
    // Ако има оцена за тренутни месец, рачуна се просек и приказује извештај
}
```

<sup>3</sup> Регуларни израз (Regex) је шаблон који служи за проналажење и обраду делова текста који прате одређену структуру. Користи се за валидирање, издвајање или замену текста у стринговима. На пример, у овом коду Regex се користи да препозна предмете, оцене и датуме у једном стрингу.

## 4. Закључак

Израда овог матурског рада омогућила ми је да на практичном примеру применим стечена знања из области база података, програмирања и рада у Unity окружењу. Током развоја апликације прошао сам кроз све фазе креирања софтверског система — од планирања структуре базе, преко имплементације фронтенда и бекенда, до тестирања и оптимизације функционалности.

Највећи изазов био је остварити ефикасну и сигурну комуникацију између Unity апликације и базе података, као и осмислити начин за организовање оцена у формату који је лако читљив и обрадив. Уз помоћ регуларних израза и JSON формата, успео сам да постигнем да подаци буду прегледни и једноставни за приказ и обраду.

Овај пројекат ми је помогао да боље разумем колико су информациони системи значајни у свакодневном животу и колико је важно водити рачуна о безбедности и интегритету података. У будућности бих ову апликацију могао унапредити додавањем могућности слања обавештења ученицима, као и аутоматизованим системом верификације налога путем електронске поште.

Сматрам да сам кроз овај рад стекао драгоцену искуство које ће ми бити од велике користи у даљем образовању и будућем раду у ИТ сектору.

# Литература

- [1] Board To Bits Games - Unity & MySQL Databases  
[https://www.youtube.com/watch?v=CO\\_DK75XOl4&list=PL5KbKbJ6Gf99mcmE1p\\_tsn0oXO1\\_vnKDI5](https://www.youtube.com/watch?v=CO_DK75XOl4&list=PL5KbKbJ6Gf99mcmE1p_tsn0oXO1_vnKDI5), 1. јун 2025. године
- [2] <https://petlja.org/sr-Latn-RS/kurs/4655/>, 1. јун 2025. године
- [3] <https://docs.unity3d.com/Manual>, 1. јун 2025. године.

Датум предаје: \_\_\_\_\_

Чланови испитне комисије:

-Председник: \_\_\_\_\_

-Испитивач: \_\_\_\_\_

-Члан комисије: \_\_\_\_\_

Коментар:

Оцена вредности матурског рада: \_\_\_\_\_

Оцена одбране матурског рада: \_\_\_\_\_

Закључна оцена: \_\_\_\_\_

Датум одбране:

\_\_\_\_\_